

## **Title: “True Fans” - National Hockey League Stats Application**

### **Introduction:**

The National Hockey League (NHL), is considered one of the “big four” professional sports leagues in North America along with NBA, NFL, and MLB. Despite the NHL having more teams than the NBA or MLB, NHL is ranked last when it comes to its revenue ranking and fan bases. Hockey is a very fast moving sport and for those who are not familiar with the game, participating in a game that chases a tiny little rubber puck on an ice skate could be overwhelming. Goal of our application is to provide more insightful information about the sport via data by bringing interesting stats about the game and the players to increase interests for people who are not necessarily a “true fan” of hockey. Our hope is that when a person engages on a website that helps them be quickly educated more about the game and the players they have a tendency to follow the league more closely, and get more out of the game when they do decide to watch. The granular key metrics like points per game, total ice time per game, power play minutes, and goals against (GA) vs. goals for (GF), are often shared during broadcast via sportscasters and hockey experts. By enabling web application users to interact with these key metrics, they would be familiarized with how performance is measured and start to speak the “Hockey Language”, and hence create more stickiness to the sports and become a true fan in no time. Finally those jargons that the announcer throws out at the audience make total sense!

### **Team Members:**

Jason Barnett, [jabarnet@seas.upenn.edu](mailto:jabarnet@seas.upenn.edu)

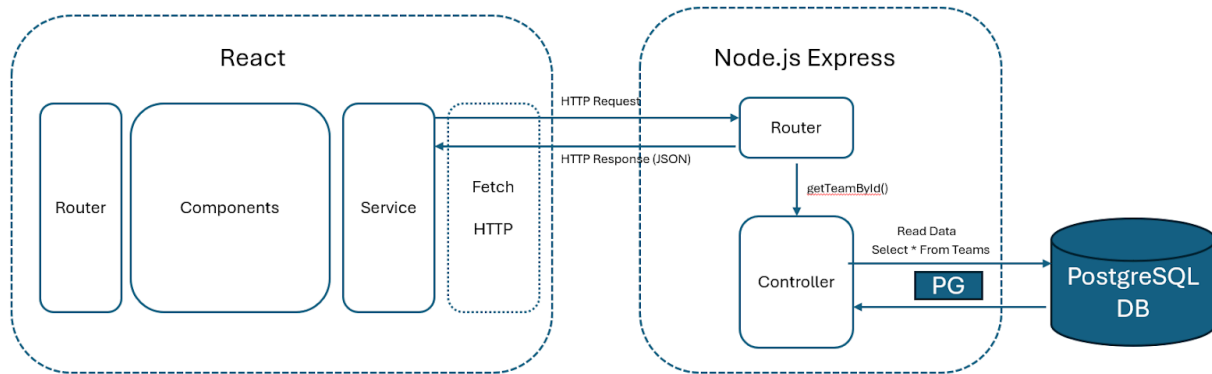
Molly O’Mara, [momara@seas.upenn.edu](mailto:momara@seas.upenn.edu)

Yuhao Yan, [yuhaoyan@seas.upenn.edu](mailto:yuhaoyan@seas.upenn.edu)

### **Architecture:**

#### **Tech Components:**

- **DB: Progres**
  - [postgres@postgres.cvwulvwoab4h.us-east-1.rds.amazonaws.com](mailto:postgres@postgres.cvwulvwoab4h.us-east-1.rds.amazonaws.com)
  - [cis550hockey](#) / [JaMoYu123](#)
- **Web Server: NodeJS**
- **Front-End Client: React**
- **Development: Visual Studio Code**
- **DataGrip DB IDE**
- **Version Control: Github**
- **NOTE: DDL file is checked in, in Github.**



**Home Page:** Homepage is set up with a clean and simple search page by design (similar to google search), not to overwhelm the first time user with tons and tons of information upfront. Instead, the search field offers freeform search terms for two of the most popular search needs, look for a player or a team. Once a search term is entered the page performs both player name search and team name search in parallel. The results are filtered dynamically as the user types in each character. Clicking on the search result takes the user to either the player page or team page depending on the user selection. The home page also provides few links to popular topics like the league standings, or pregame info of any two team's match up.

**League Standings Page:** League Standings page offers a full summary of each team's ranking based on their performance for the year for each season. The list is sorted based on points which is calculated as 2 points for any win including an overtime win, 1 point for an overtime loss, and 0 point for loss and aggregated to come up with the total points earned for each season and ordered by the points in descending order. The seasons dropdown allows a user to view any past year's ranking. The League Standings page also offers hierarchical drill down features of the site. Upon selecting a team from the standings list, a user can view detailed team stats for the selected team and its roster for each season.

**Team Details and Roster Page:** Team Details and Roster page provides a summary of the team's performance for a season with key metrics like total points, winning percentage, total goals for and total goals against. The rosters section provides a list of all the players played for a given season for the selected team, their performance metrics and contributions ordered by their total points earned (1 point for a goal and 1 point for a primary or secondary assist). Similarly a user can select any season of interest from the drop down menu and the data refreshes to reflect the selected year for the team stats and its registered roster for that year.

**Player Profile Page:** Player Profile page provides an interesting summary stats of individual player's performance for each season played in the league. In addition to key metrics like goals, assists, and points, it also displays some interesting calculated metrics like shooting percentages, often used for measuring the player's skills and contribution, and average total ice time per game which is a clear indicator or how valuable "core" player the particular play is for the team. Highest value often is considered MVP of the team and anything above 20 min would

be considered an elite league player. The heatmap of the shots is a nice visual enhancement to the application showing where the shots were made from on the ice by plotting x and y coordinates of the ice and number of shots made from that area. More advanced analysis can be performed visually from the heatmap by comparing all the shots made for a given year and spotting trends over time as the player develops his career over the years in the league.

**Game Day, Head to Head Page:** This page is meant to show the tale of the tape between two matchups from any two teams in the league. A pregame insight is another great way to get the fans going prior to an upcoming match between two teams. The page allows users to select any two teams via a simple search UI and upon selecting two teams, it provides side by side comparisons of the past matchups for the latest season. The table shows the result details of each of the match ups of the season. If there is a dominant team, a “trophy” icon is awarded. It’s a fun way to display who is the “favorite” and who is the “underdog” going into the next match up.

**Goal Predictor Page:** This page provides deeper analysis data on each player or a team based on expected goals (calculated predictive metric from the MoneyPuck database) and actual goals recorded from the NHL games database. The goal differentials table indicates whether the player or the team is playing better or worse than its expected performance predictions. Only the true fans care.

## Data:

We leveraged two complementary datasets to show comprehensive hockey analytics. The first dataset, downloaded from Kaggle (<https://www.kaggle.com/datasets/martinellis/nhl-game-data>), contains official per-game statistics for players, teams, and games from the 2000-2001 season to the 2019-2020 season. It includes player/team/game information referenced by their respective IDs, along with game plays categorized into events such as shots, goals, penalties, etc. The second dataset from MoneyPuck (<https://moneypuck.com/data.htm>) provides season-averaged data for players and teams under different situations (5on5, 5on4, 4on5), featuring both factual attributes like number of goals and shots, as well as calculated statistics such as expected goals, low/high danger goals, and flurry-adjusted expected goals.

These datasets power distinct but interconnected features in the application. The NHL dataset provides the core game experience features, including team stats (/teams/team\_id), team standings (/standings), player profile info (/players/player\_id/profile) and team head-to-head games comparisons (/teamheadtohead). Meanwhile, the MoneyPuck dataset enables more advanced analytical features through endpoints like /goalsdifference and /playergoalsdifference, which compare expected versus actual goals to reveal shooting efficiency patterns. We also utilized NHL's detailed x/y coordinates from game plays and grouped shots into grid locations and aggregates by type (Shot, Goal, Missed Shot) which provides visualizations of shooting patterns and effectiveness zones (/players/heatmap).

For preprocessing the NHL dataset, we first checked for null values in each table and determined whether they were expected. For example, in the `game_plays` table, the (x, y) coordinates indicate where on the field an event occurred. These values are only relevant for certain event types like shots or blocks, so it's expected that they are null for other types. We did not modify these expected nulls. However, we found some unexpected nulls, such as missing heights for a few players or missing shot counts for a few games. These were rare (e.g., only 3 out of 3952 players missing height and 8 out of 48,000 team-game entries missing shot data), so we chose not to remove them. Some fields were also null for older seasons due to incomplete data collection, but these columns were not used in our analysis, so they did not affect our work. We did not fill any nulls using column averages or other imputation methods. Lastly, we removed duplicate entries from all tables—up to 10% in some cases, as detailed in the SQL DDL files.

For preprocessing the Money puck dataset, we first combined the yearly average data tables into a single table. We then standardized the team abbreviations and team ID mappings to match those used in the NHL dataset. After checking for null values, we found none in the Money puck data.

#### **Clean and pre-process strategy.**

1. Review the major datasets and its original data schema
2. Identify the primary keys (i.e. player's unique id, team's unique id) and review potential joins. Cross reference player name and team name between two datasets, and add player/team ids from NHL dataset to money puck dataset.
3. Identify the redundant attributes and determine which ones to keep and which ones to remove
4. Import the data model from each dataset
  - Removal of rows with missing values and null values
  - Replacement of missing values with default values
5. Normalize the dataset

#### **SQL DDL for creating the database**

\*DDL files are stored in our GitHub repository.

#### **Database:**

We uploaded the preprocessed tables from the two datasets into a PostgreSQL database we created using SQL DDL code (see on Github). A small fraction of instances were not successfully uploaded due to missing foreign key references (for example, a player's season stat has `player_id` that does not exist in the `PLAYER_INFO` table). As mentioned previously, most of Money puck data were successfully imported with `player_id` and `team_id` referencing the same in the `PLAYER_INFO` and `TEAM_INFO` table in the NHL dataset.

ER diagrams (Please see appendix)

There are in total 26.3k instances in GAME, 56.7k instances in GAME\_GOALIE\_STATS, 149k instances in GAME\_GOALS, 106k instances in GAME\_OFFICIALS, 248k instances in GAME\_PENALTIES, 5.05m instances in GAME\_PLAYS, 7.59m instances in GAME\_PLAY\_PLAYERS, 137k instances in GAME\_SCRATCHES, 11.9m instances in GAME\_SHIFTS, 946k instances in GAME\_SKATER\_STATS, 52.6k instances in GAME\_TEAM\_STATS, 3926 players and 33 teams included in the NHL dataset. The Money puck dataset includes 363 team/season data sets, 10620 skater/season data sets, and 1079 goalie/season data sets with each split into 5 game situations.

The database schema follows 3NF and BCNF normalization rules across all tables. Each table has primary keys - either single columns like team\_id in TEAM\_INFO or multiple columns like (game\_id, team\_id) in GAME\_TEAMS\_STATS. All non-key columns depend directly on their table's primary key, with no dependencies where one non-key column depends on another non-key column. For example, in PLAYER\_INFO, attributes like name, nationality, and physical characteristics all depend solely on player\_id. In the statistical tables of the Money puck dataset like GAME\_SKATER\_STATS and SEASON\_TEAM\_STATS, all performance metrics depend completely on their composite keys. Different tables are linked through foreign key references without redundant columns. For example, GAME\_PLAYER\_STATS have three foreign keys: (game\_id, team\_id, player\_id) and each of them links to an instance in the GAME, TEAM\_INFO, PLAYER\_INFO tables; GAME\_PLAYER\_STATS only contains a player's stats in that particular game such as shots, goals, etc whereas the information of the player like last name and first name is stored in PLAYER\_INFO. The only exception is we retained last names and first names in the SEASON\_PLAYER\_STATS table of the Money puck dataset. Although the Money puck dataset uses the same player\_id field as the NHL dataset, we wanted to ensure the names correspond across the two datasets and having names in the Money puck tables provide a simple way of checking. There are no partial dependencies in tables with composite keys - each stored value needs the entire key to be uniquely identified.

### **Queries:**

### **Key use cases:**

League Standings, Team Roster, Individual Player Profile, Team Search, Player Search, Player Season Stats, Team Season Stats, Team Match Up (head to head), and Player Performance Heatmap

### Query Example: League Standings

This query is mainly used for generating the league's team ranks based on total points earned for each season. This is achieved through layers of aggregation DB techniques. The games data is scanned for all the regular season games and aggregate each team's win in both home and away matches in regular periods vs versus losing in overtime. This is because the points system calculator regular period wins as 2 points and overtime loss is still worth 1 point.

```
WITH nhl_games AS (
    SELECT g.home_team_id,
           g.away_team_id,
           g.outcome,
           g.home_goals,
           g.away_goals,
           g.season
    FROM game AS g
    WHERE g.type = 'R'
    AND g.season = $1
),
nhl_standings AS (
    SELECT ng.season,
           ng.home_team_id AS team,
           'home' AS home_away,
           ng.home_goals AS goals_for,
           ng.away_goals AS goals_against,
           CASE WHEN ng.outcome = 'home win REG' THEN 2
                WHEN ng.outcome = 'home win OT' THEN 2
                WHEN ng.outcome = 'away win OT' THEN 1
                ELSE 0 END AS points
    FROM nhl_games AS ng
    UNION ALL
    SELECT ng.season,
           ng.away_team_id AS team,
           'away' AS home_away,
           ng.away_goals AS goals_for,
           ng.home_goals AS goals_against,
           CASE WHEN ng.outcome = 'away win REG' THEN 2
                WHEN ng.outcome = 'away win OT' THEN 2
                WHEN ng.outcome = 'home win OT' THEN 1
                ELSE 0 END AS points
    FROM nhl_games AS ng
)
SELECT ns.season,
       ns.team AS team_id,
       t.shortname,
```

```

t.teamname,
COUNT(*) AS games_played,
SUM(CASE WHEN points = 2 THEN 1 ELSE 0 END) AS wins,
SUM(CASE WHEN points = 0 THEN 1 ELSE 0 END) AS losses,
SUM(CASE WHEN points = 1 THEN 1 ELSE 0 END) AS ot_losses,
SUM(goals_for) AS goals_for,
SUM(goals_against) AS goals_against,
SUM(goals_for) - SUM(goals_against) AS plus_minus,
SUM(points) AS points
FROM nhl_standings AS ns
JOIN team_info t ON ns.team = t.team_id
GROUP BY ns.season, ns.team, t.shortname, t.teamname
ORDER BY points DESC, wins DESC, plus_minus DESC

```

### Query Example: Head to Head Matchup Prediction

This query scans the games\_played data for a given season between any two teams and analyzes games played against each other based on the match results. Generates additional calculated metrics such as average goals per match, average shots per game and power play goal percentages against each other.

Part I:

WITH matchup\_games AS (

```

    SELECT game_id
    FROM game
    WHERE season = $3
    AND ((home_team_id = $1 AND away_team_id = $2) OR (home_team_id = $2 AND
away_team_id = $1))
),
team1_stats AS (
    SELECT
        t.teamname,
        COUNT(*) as games_played,
        SUM(CASE WHEN gts.won THEN 1 ELSE 0 END) as wins,
        ROUND(AVG(gts.goals), 2) as avg_goals,
        ROUND(AVG(gts.shots), 2) as avg_shots,
        ROUND(SUM(gts.powerPlayGoals) * 100.0 / NULLIF(SUM(gts.powerPlayOpportunities),
0), 1) as pp_percentage
    FROM game_teams_stats gts
    JOIN team_info t ON gts.team_id = t.team_id
    JOIN matchup_games mg ON gts.game_id = mg.game_id
    WHERE gts.team_id = $1
    GROUP BY t.teamname
),

```

```

team2_stats AS (
  SELECT
    t.teamname,
    COUNT(*) as games_played,
    SUM(CASE WHEN gts.won THEN 1 ELSE 0 END) as wins,
    ROUND(AVG(gts.goals), 2) as avg_goals,
    ROUND(AVG(gts.shots), 2) as avg_shots,
    ROUND(SUM(gts.powerPlayGoals) * 100.0 / NULLIF(SUM(gts.powerPlayOpportunities),
0), 1) as pp_percentage
  FROM game_teams_stats gts
  JOIN team_info t ON gts.team_id = t.team_id
  JOIN matchup_games mg ON gts.game_id = mg.game_id
  WHERE gts.team_id = $2
  GROUP BY t.teamname
)
SELECT
  t1.teamname as team1_name,
  t2.teamname as team2_name,
  t1.games_played,
  t1.wins as team1_wins,
  t2.wins as team2_wins,
  t1.avg_goals as team1_avg_goals,
  t2.avg_goals as team2_avg_goals,
  t1.avg_shots as team1_avg_shots,
  t2.avg_shots as team2_avg_shots,
  t1.pp_percentage as team1_pp_percentage,
  t2.pp_percentage as team2_pp_percentage
FROM team1_stats t1, team2_stats t2`

```

PART II:

```

SELECT
  g.game_id,
  g.date_time_GMT,
  t1.teamname as home_team,
  t2.teamname as away_team,
  g.home_goals,
  g.away_goals,
  g.outcome
FROM game g
JOIN team_info t1 ON g.home_team_id = t1.team_id
JOIN team_info t2 ON g.away_team_id = t2.team_id
WHERE g.season = $3
  AND ((g.home_team_id = $1 AND g.away_team_id = $2) OR (g.home_team_id = $2 AND
g.away_team_id = $1))

```



ORDER BY g.date\_time\_GMT DESC

### Query Example: Player Shooting Heatmap

This query is used to provide a heatmap component with a series of x and y coordinates of where the shot was made on the arena and number of shots made from that x,y coordinate into essentially (x coordinate, y coordinate, values) tuples. Calculated x and y coordinates were generated from joining games\_plays, player\_info, and team.

```
SELECT
    CONCAT(pi.firstname, ' ', pi.lastname) AS player_name,
    g.season,
    ti.teamname,
    gp.event AS event_type,
    (gp.x / 5) * 5 AS x_coord,
    (gp.y / 5) * 5 AS y_coord,
    COUNT(*) AS shot_count
FROM game_plays gp
JOIN game_plays_players gpp ON gp.play_id = gpp.play_id AND gpp.playerType = 'Shooter'
JOIN player_info pi ON gpp.player_id = pi.player_id
JOIN game g ON gp.game_id = g.game_id
JOIN team_info ti ON gp.team_id_for = ti.team_id
WHERE gpp.player_id = $1
AND g.season = $2
AND gp.event IN ('Shot', 'Goal', 'Missed Shot')
AND gp.x IS NOT NULL AND gp.y IS NOT NULL
GROUP BY
    pi.firstname, pi.lastname,
    g.season,
    ti.teamname,
    gp.event,
    (gp.x / 5) * 5,
    (gp.y / 5) * 5
ORDER BY shot_count DESC
```

### Query Example: Expected Goals vs. Actual Goals

The query combines low, medium and high danger chances of scoring goals from expected goals dataset as well as combining actual goals scored from low danger, medium danger, and high danger shots recorded from actual goals dataset. Goal differentials were calculated by comparing the two results. The query is performed at both individual player level as well as aggregated team level comparisons to track if a team or a player overperformed or underperformed the expectations.

{Player Level}

```

SELECT
    ss.player_id,
    pi.firstname || ' ' || pi.lastname AS player_name,
    ss.team,
    ti.teamname,
    ss.season,
    ss.position,
    ss.games_played,
    ROUND(SUM(ss.i_f_lowDangerxGoals + ss.i_f_mediumDangerxGoals +
ss.i_f_highDangerxGoals)::numeric, 2) AS expected_goals,
    ROUND(SUM(ss.i_f_lowDangerGoals + ss.i_f_mediumDangerGoals +
ss.i_f_highDangerGoals)::numeric, 2) AS actual_goals,
    ROUND((SUM(ss.i_f_lowDangerGoals + ss.i_f_mediumDangerGoals +
ss.i_f_highDangerGoals) -
    SUM(ss.i_f_lowDangerxGoals + ss.i_f_mediumDangerxGoals +
ss.i_f_highDangerxGoals))::numeric, 2) AS goals_difference
FROM season_skater_stats ss
JOIN player_info pi ON ss.player_id = pi.player_id
JOIN team_info ti ON ss.team = ti.abbreviation
WHERE ss.situation = 'all'

```

{Team Level}

```

SELECT
    ss.team,
    ti.teamname,
    ss.season,
    ROUND(SUM(ss.i_f_lowDangerxGoals + ss.i_f_mediumDangerxGoals +
ss.i_f_highDangerxGoals)::numeric, 2) AS expected_goals,
    ROUND(SUM(ss.i_f_lowDangerGoals + ss.i_f_mediumDangerGoals +
ss.i_f_highDangerGoals)::numeric, 2) AS actual_goals,
    ROUND((SUM(ss.i_f_lowDangerGoals + ss.i_f_mediumDangerGoals +
ss.i_f_highDangerGoals) -
    SUM(ss.i_f_lowDangerxGoals + ss.i_f_mediumDangerxGoals +
ss.i_f_highDangerxGoals))::numeric, 2) AS goals_difference
FROM season_skater_stats ss
JOIN team_info ti ON ss.team = ti.abbreviation
WHERE ss.situation = 'all';

```

```

if (team_id) {
    query += ` AND ti.team_id = ${paramIndex}`;
    params.push(team_id);
    paramIndex++;
}

```

```

if (team_name) {
  query += ` AND (LOWER(ti.teamname) LIKE LOWER(${paramIndex}) OR
LOWER(ti.abbreviation) LIKE LOWER(${paramIndex}))`;
  params.push(`%${team_name}%`);
  paramIndex++;
}

query += `
  GROUP BY ss.team, ti.teamname, ss.season
  ORDER BY goals_difference DESC`

```

### Query Example: Team Performance

This query creates performance metrics for teams in the 2019-2020 season. It first calculates shooting statistics in a CTE, then joins this with the standings to combine traditional stats (wins, points) with shooting efficiency. The results are ordered by winning percentage to show the most successful teams first.

```

WITH shooting_stats AS (
  SELECT T.team_id,
         T.teamName AS teamname,
         SUM(GTS.goals) AS total_goals,
         SUM(GTS.shots) AS total_shots,
         COUNT(DISTINCT GTS.game_id) AS games_played,
         ROUND(SUM(GTS.goals) * 1.0 / NULLIF(SUM(GTS.shots), 0), 4) AS
shooting_percentage
  FROM game_teams_stats GTS
  JOIN team_info T ON GTS.team_id = T.team_id
  JOIN game G ON GTS.game_id = G.game_id
  WHERE G.season = 20192020
  GROUP BY T.team_id, T.teamName
)
SELECT
  ts.teamname AS team_name,
  ROUND((ts.wins * 1.0 / NULLIF(ts.games_played, 0)), 3) AS winning_percentage,
  ss.shooting_percentage,
  ts.points AS total_points,
  ts.goals_for AS goals_for,
  ts.goals_against AS goals_against,
  ss.total_goals,
  ss.total_shots
FROM team_standings ts
JOIN shooting_stats ss ON ts.team_id = ss.team_id
WHERE ts.season = 20192020

```

ORDER BY winning\_percentage DESC, shooting\_percentage DESC, total\_points DESC

### Query Example: Team Roster

Team roster dataset is generated with key performance metrics from selecting all the players recorded for a given season for the team by joining game\_skater\_stats with player\_info and left join season\_skater\_stats.

```
SELECT DISTINCT g.player_id, p.firstname, p.lastname, p.primaryPosition,
               s.games_played, s.i_f_goals AS goals,
               (s.i_f_primaryassists + s.i_f_secondaryassists) AS assists,
               s.i_f_points AS points
FROM game_skater_stats g
JOIN player_info p ON p.player_id = g.player_id
LEFT JOIN season_skater_stats s ON s.player_id = g.player_id AND s.season = $2 AND
s.situation = 'all'
WHERE g.team_id = $1
AND g.game_id IN (SELECT game_id FROM game WHERE season = $2)
ORDER BY points DESC NULLS LAST
```

### Performance evaluations:

Head to Head Page Performance Optimization:

Input is getting matching games with two matching teams for all seasons from the game table and getting aggregated team stats for those matching games from the game\_team\_stats table. Without any improvement: 1s 304ms. We added an index on game(season, home\_team\_id, away\_team\_id) which we hoped to improve look up speed for games in specific season involving two given teams. We also added an index on game\_teams\_stats(team\_id, game\_id) which we hoped to help querying a team's stats in a specific game faster. These two indexes led to marginal improvement with 1s 121ms. This is possibly due to the fact that this query involves many aggregation functions like COUNT and SUM, as well as multiple joins, which do not improve with indexes.

Team Expected Goals vs Actual Goals:

Input is aggregated expected goals and actual goals for each team from the season\_team\_stats. Without any improvement: 1s 939ms. We created a materialized view which has the expected goals vs actual goals pre-computed for each team each season. This allows the query from the client side to directly look up the goal difference value from the mv, without needing to compute it each time. With mv, the same query took 985 ms.

Team Standings Optimization:

Input is regular season game outcomes and involved teams, from which we computed cumulated points for each team in a season. Without any improvement: 2s 405ms. We added

an index on game(season, type), which allows looking up games in each season and finding regular season games much faster. We added an index on game(outcome) which allows determining the result of each game and computing points for teams in that game much faster. With indexes, the same query improves to 1s 160ms.

#### Shooting heat map Optimization:

Input is game play events that are shots for each player from game\_plays and game\_plays\_players table. From the shots which have x and y coordinates we bin them into 5x5 grids on the rink and cumulate the shots each player made in each zone for a season. Without any improvement: 7s 524ms. We added an index on game\_play(events) filtered by “shots”, “goals” and “missed shots”. This allows the DB to only look at events that are in those categories relevant to shot heatmap. We added an index on game\_play(x,y), which quickly finds game\_play events whose coordinates are valid. We also added an index on game (season) to quickly locate game\_plays in games of a particular season. After adding indexes, the same query improves to 3s 76ms.

#### Technical Challenges:

- General coding, troubleshooting and bug tracking given limited experience in REACT framework.
- Just getting the overall framework to work from backend to frontend took some time to master.
- Errors caused by some differences in how PostgreSQL handles certain functions like double precision attributes and applying math functions like ROUND were hard to detect.
- We could have done a better job in code management in general with GitHub leveraging true code checkin and checkout best practices to avoid possible overwriting of code or version changes. Lessons learned.
- Adding some more fancy visual components were not “free” and only available in “pro” version. The heatmap was quite challenging to make it work, and full credits to **Edward**, one of our members for his effort to generate this awesome visual heat map that actually looks like an ice hockey rink.
- Parameter passing technique via URL for inserting various ids as part of the URL string took some time to master in the beginning.

#### Extra Credits and Bonus Features Implemented:

**Visual Enhancement:** Visual enhancement with sorting function and color coded values for predicted vs. actual - specific or range of players, by each season, for each team displayed on sortable tables

**Deeper Analysis:** More granular player and team level analysis on expected goals vs. actual goals. Expected goal percentages were driven by high danger chances, medium danger changes, and low danger chances recorded and compared against actual for differences.

**Heatmap:** Heat map showing zones, displayed on player profile with amount of shots taken from. Where players take shots from to generate high danger chances of scoring.

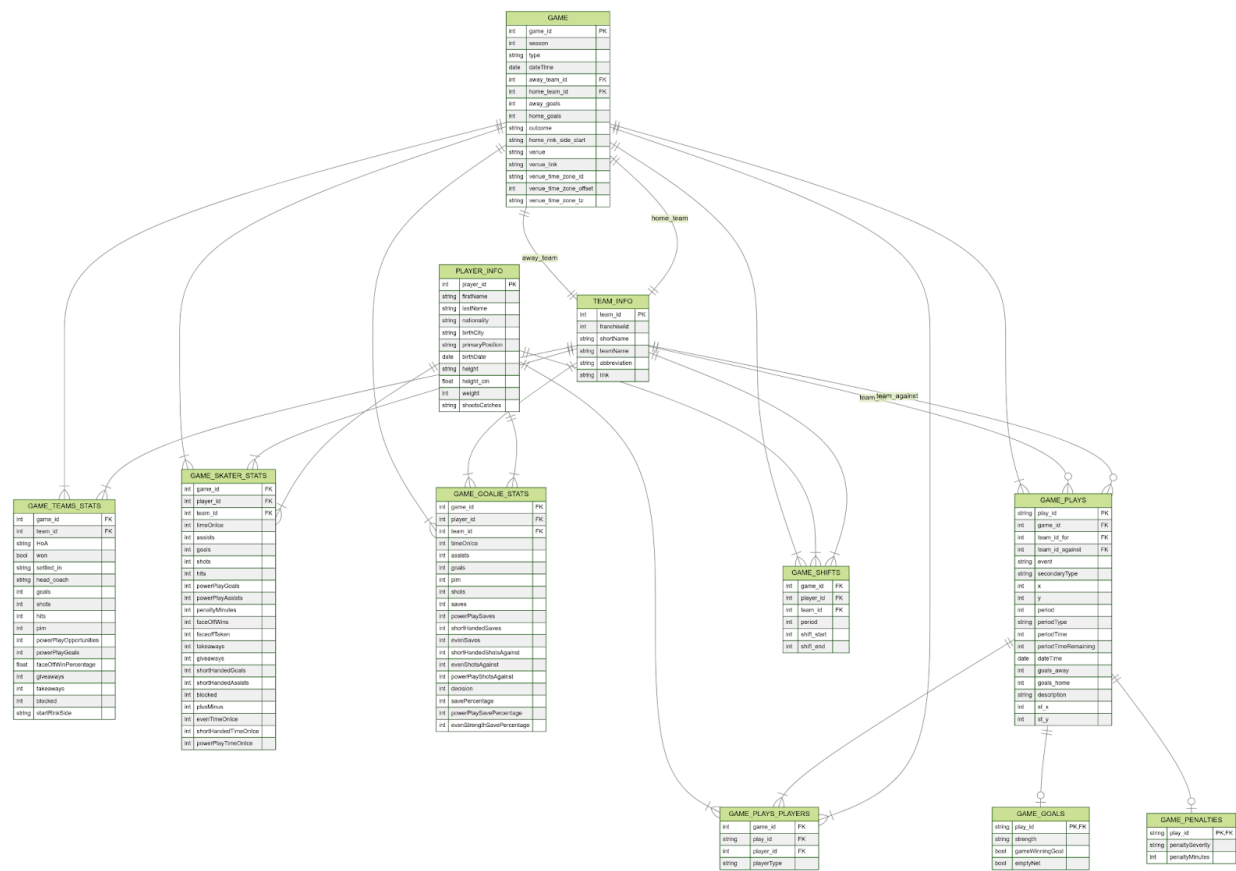
**Detailed Profile Page with Seasons Stats:** Detailed player profiles highlights key metrics career with following enhancements:

- High danger shots/ 5-alarm shots
- Shooting average vs. Scoring %
- Season summary for all season played by the player

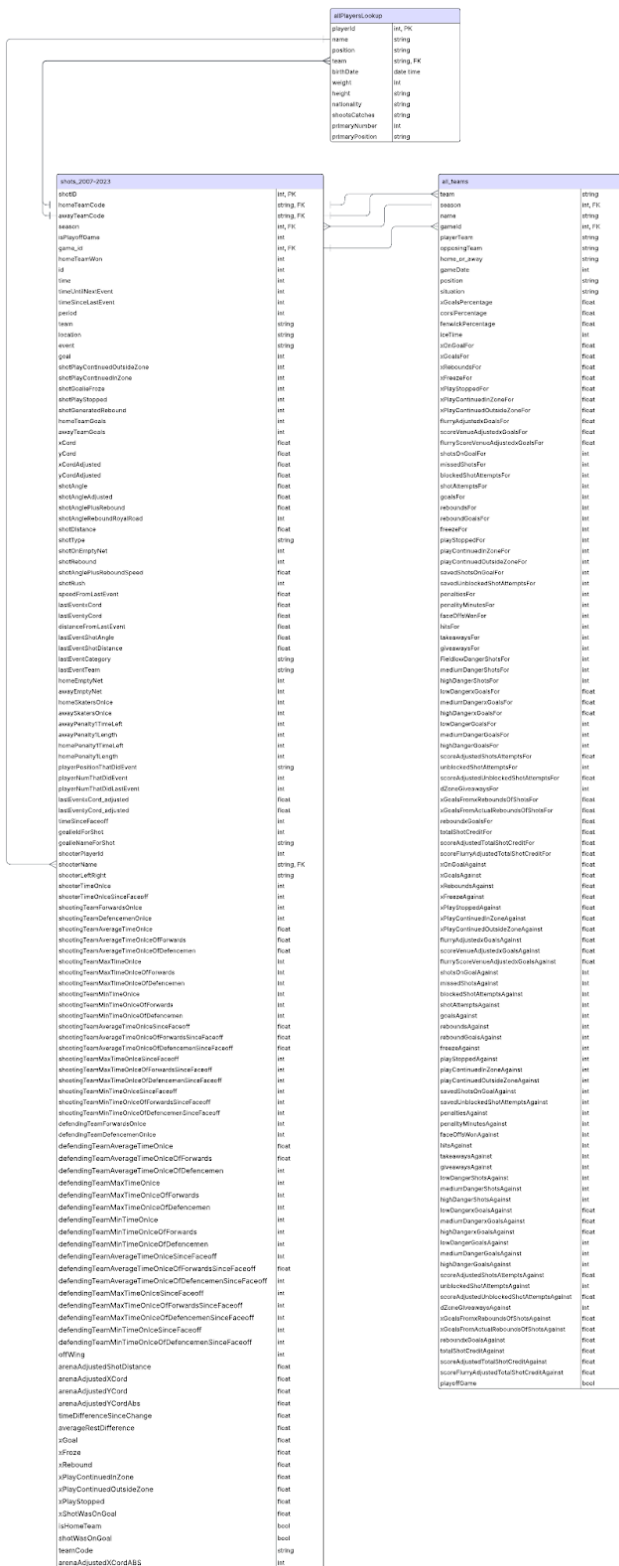
**Pre-game Matchup for Game Day:** Game day match up analysis, selecting two teams and providing comparison charts in a “tale of the tape” format and winner prediction with “trophy” award if winner is deterministic by the wins.

Appendix A. ER Diagrams

NHL dataset:



### MoneyPuck dataset:





## Appendix B. Additional Queries Used

### Query: Individual Player Profile

The query is used to populate the single player's profile including its biography information and the player's key performance metrics for each season played in a table.

```
SELECT
    p.firstname,
    p.lastname,
    p.nationality,
    p.birthcity,
    p.primaryposition,
    p.birthdate,
    p.height,
    p.weight,
    p.shootscatches,
    p.player_id
FROM player_info p
WHERE p.player_id = $1
```

```
SELECT
    season,
    games_played,
    i_f_goals AS goals,
    i_f_primaryassists + i_f_secondaryassists AS assists,
    i_f_points AS points,
    i_f_shotsOnGoal AS shots,
    ROUND(CAST((i_f_goals * 1.0 / NULLIF(i_f_shotsOnGoal, 0)) * 100 AS numeric), 1) AS
shooting_percentage,
    icetime / games_played AS avg_time_on_ice
FROM season_skater_stats
WHERE player_id = $1 AND situation = 'all'
ORDER BY season
```

### Query: Get Teams

```
SELECT *
FROM TEAM_INFO
ORDER BY teamName
```

### Query: Get Team by ID

```
SELECT *
FROM TEAM_INFO
WHERE team_id = $1
```

**Query: Get Player**

```
SELECT *  
  FROM PLAYER_INFO  
  ORDER BY lastName, firstName  
  LIMIT $1 OFFSET $2
```

**Query: Get Player by ID**

```
SELECT *  
  FROM PLAYER_INFO  
  WHERE player_id = $1
```

**Query: Get Games**

```
SELECT g.*,  
       ht.teamName as home_team_name,  
       at.teamName as away_team_name  
  FROM GAME g  
  JOIN TEAM_INFO ht ON g.home_team_id = ht.team_id  
  JOIN TEAM_INFO at ON g.away_team_id = at.team_id  
`;  
;
```

```
const params = [];
```

```
let paramCount = 1;
```

```
if (season) {  
  query += ` WHERE g.season = ${paramCount}`;  
  params.push(season);  
  paramCount++;  
}
```

```
query += ` ORDER BY g.date_time_GMT DESC  
          LIMIT ${paramCount} OFFSET ${paramCount + 1}
```

**Query: Get Game by ID**

```
SELECT g.*,  
       ht.teamName as home_team_name,  
       at.teamName as away_team_name  
  FROM GAME g  
  JOIN TEAM_INFO ht ON g.home_team_id = ht.team_id  
  JOIN TEAM_INFO at ON g.away_team_id = at.team_id  
  WHERE g.game_id = $1
```

**Query: Get Game Plays**

```

SELECT p.*,
       t1.teamName as team_for_name,
       t2.teamName as team_against_name
FROM GAME_PLAYS p
LEFT JOIN TEAM_INFO t1 ON p.team_id_for = t1.team_id
LEFT JOIN TEAM_INFO t2 ON p.team_id_against = t2.team_id
WHERE p.game_id = $1
ORDER BY p.period, p.periodTime

```

#### Query: Get Team Stats

```

SELECT *
FROM SEASON_TEAM_STATS
WHERE team_id = $1 AND situation = $2
`;

```

```
const params = [team_id, situation];
```

```

if (season) {
  query += ` AND season = $3

```

#### Query: Get Player Season Stats

```

SELECT primaryPosition FROM PLAYER_INFO WHERE player_id = $1`,
[player_id],
(err, posData) => {
  if (err) {
    console.error(err);
    return res.status(500).json({ error: 'Internal server error' });
  }

  if (posData.rows.length === 0) {
    return res.status(404).json({ error: 'Player not found' });
  }

```

```

const isGoalie = posData.rows[0].primaryPosition === 'G';
const table = isGoalie ? 'SEASON_GOALIE_STATS' : 'SEASON_SKATER_STATS';

```

```

let query = `
SELECT *
FROM ${table}
WHERE player_id = $1 AND situation = $2
`;

```

```
const params = [player_id, situation];
```

```

if (season) {
  query += ` AND season = $3

```

### Query: Team Performance by Situation

```

SELECT
  t.teamname,
  gts.HoA,
  COUNT(*) as games,
  SUM(CASE WHEN gts.won THEN 1 ELSE 0 END) as wins,
  SUM(CASE WHEN NOT gts.won THEN 1 ELSE 0 END) as losses,
  ROUND(SUM(CASE WHEN gts.won THEN 1 ELSE 0 END)*100.0/NULLIF(COUNT(*), 0),
1) as win_percentage,
  ROUND(AVG(gts.goals), 2) as avg_goals_for,
  ROUND(AVG(gts.shots), 2) as avg_shots_for,
  ROUND(AVG(gts.powerPlayGoals), 2) as avg_pp_goals,
  ROUND(AVG(gts.powerPlayOpportunities), 2) as avg_pp_opportunities,
  ROUND(AVG(gts.powerPlayGoals) * 100.0 / NULLIF(AVG(gts.powerPlayOpportunities), 0),
1) as pp_percentage,
  ROUND(AVG(gts.faceOffWinPercentage), 1) as avg_faceoff_win_pct
FROM game_teams_stats gts
JOIN team_info t ON gts.team_id = t.team_id
JOIN game g ON gts.game_id = g.game_id
WHERE gts.team_id = $1
AND g.season = $2
GROUP BY t.teamname, gts.HoA
ORDER BY win_percentage DESC

```

### Query: Search Player & Team

```

SELECT DISTINCT p.player_id, p.firstname, p.lastname, p.primaryposition, p.nationality,
p.birthdate, p.height, p.weight
FROM player_info p
`;

```

```

if (team || season) {
  query += `
    JOIN game_skater_stats gs ON p.player_id = gs.player_id
    JOIN game g ON gs.game_id = g.game_id
  `;
}

```

```

let whereClause = [];

```

```

const params = [];
let paramIndex = 1;

if (name) {
  whereClause.push(`p.firstname ILIKE ${paramIndex} OR p.lastname ILIKE
${paramIndex}`);
  params.push(`%${name}%`);
  paramIndex++;
}

if (position) {
  whereClause.push(`p.primaryposition = ${paramIndex}`);
  params.push(position);
  paramIndex++;
}

if (nationality) {
  whereClause.push(`p.nationality = ${paramIndex}`);
  params.push(nationality);
  paramIndex++;
}

if (team) {
  whereClause.push(`gs.team_id = ${paramIndex}`);
  params.push(team);
  paramIndex++;
}

if (season) {
  whereClause.push(`g.season = ${paramIndex}`);
  params.push(season);
  paramIndex++;
}

if (whereClause.length > 0) {
  query += ` WHERE ` + whereClause.join(' AND ');
}

query += ` ORDER BY p.lastname, p.firstname

```

### Query: Search Teams

```

SELECT *
  FROM team_info
 WHERE teamname ILIKE $1 OR shortname ILIKE $1 OR abbreviation ILIKE $1

```

ORDER BY teamname